

Problem Understanding

Battery State of Charge (SoC) Estimation & Remaining Charging-Time Prediction Using Machine Learning

1. Technical Problem Statement & Background

The efficient use of electric vehicles (EV) and battery energy storage systems (BESS) requires a sophisticated Battery Management System (BMS), which ensures safety, longevity, and proper scheduling of charging cycles. The BMS is expected to determine the two main state variables and provide them to both the driver and the fleet management system:

1. State of Charge (**SoC**): The amount of charge left expressed as a percentage of the maximum charge.
2. Remaining Charging Time: The time remaining until reaching the fully charged state (or a certain SoC level).

SoC State of Charge of Battery and Remaining Time for charging the battery cannot be measured physically with the help of any onboard sensor while in use. * Electrochemical Complexity: The dynamics of the internal state of the chemistry—like lithium ion concentration dynamics, SEI formation, dynamics of internal resistance, and thermal polarization effects—are all happening inside a sealed battery cell. * Nonlinear Dynamics: The dependency of internal state on terminal voltage depends nonlinearly on various parameters like temperature, C-rate (discharge/charge), aging (SoH), and hysteresis effect. * Sensory Limits: Onboard hardware sensors can measure only the following three variables:

Terminal Voltage (V) Current (I) Surface Temperature (T).

2. Input Parameter & Output Target Identification



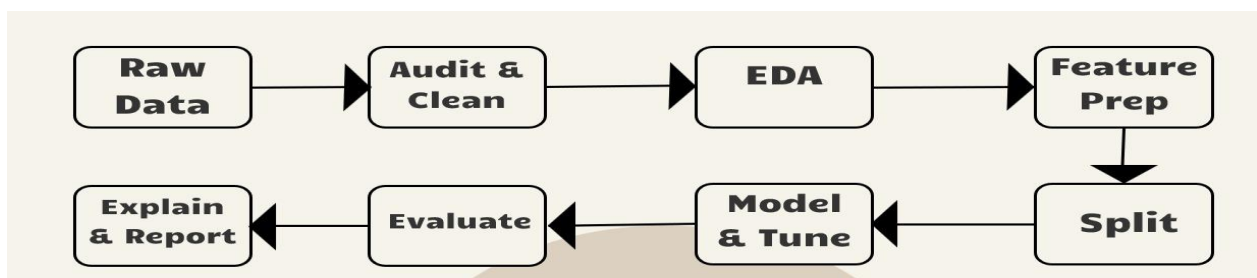
Input Feature Breakdown

- a. Battery Voltage
- b. Charging Current.
- c. Charging Power
- d. Battery
- e. Historical Charging Information
-

Output Target Identification

- Target 1 — Battery State of Charge
- Target 2 — Remaining Charging Time

3. End-to-End Software Pipeline & Success Criteria



- Raw Data: Sensor logs over time for each charging cycle, which can include multiple cycles at varying temperatures and rates of charging.
- Audit & Clean: Identification and management of sensor noise/drift issues, missing/duplicate rows, and schema differences between charging cycles.
- EDA: Exploration of the distribution of features, SoC range covered, and charging session duration variability to understand what we're working with.
- Feature Engineering: Creation of per-row and per-session features that don't rely on statistics over sessions (e.g. rates of change, cumulated charge, etc.)
- Split: Splitting by charging cycle, not per-row, so that no near-future of a session is leaked into training.
- Model & Tune: Model linear baseline, then Random Forest and XGBoost as the primary models to compare against; ANN to be explored if trees don't capture all data.
- Evaluate: Evaluate candidates using MAE, RMSE and R^2 scores separately for each target: SoC and remaining time to completion.
- Explain & Report: Reporting feature importance per target, and detection of where the model makes mistakes by phase/SoC ranges.

Success Criteria Definitions

- Accuracy: MAE, RMSE and R^2 calculated for SoC and remaining charging time respectively, beating the linear model's baseline accuracy by a pre-agreed margin.
- Comparison of ML models: Comparison of performance between Random Forest and XGBoost as the main models based on identical, session-based splits, while ANN results are used only in the case that the tree models underfit.
- Leakage-proof evaluation: Checking that session-based (and not row-based) splitting prevents leakage of the future of the same session to the training set.
- Robustness to operational conditions: Checking error by charging phases and SoC bands instead of reporting a single average figure, as required in the project objective to explain when the models fail.
- Interpretability: Target-specific feature importance provided and easy to interpret for both SoC and remaining time prediction models.
- Reproducibility: Data cleaning and EDA steps are executed from end to end and provide reproducible results

4. Technical Questions & Assumptions List for Clarification

1. What is the sampling/logging frequency within a session, and is it fixed over all the sessions and batteries?
2. What is the source of the dataset and its current availability. Is it proprietary telemetry already provided, a public benchmark, or something to be collected during this internship phase?
3. Since SoC coverage is unlikely to be spread out uniformly across the 0–100% range, how should we handle cases where certain bands were undersampled? Should we attempt to resample them, apply some weights, or just report performance there as lower?
4. Will ground-truth SoC be provided as a labeled column (e.g. from some reference BMS or lab-grade Coulomb counter)? Or do we have to infer it from another source before using it for training?

5. Are session metadata (e.g. temperature, C-Rate, chemistry/capacity) recorded alongside the telemetry, and should we treat them as categorical/numerical features or attempt to infer them from the raw sensors?
6. Since time-to-go is defined implicitly as a function of time within a session, what defines a session's end? Does it occur when the SoC reaches 100%, current drops below a certain threshold, or a fixed time allotment is exhausted (e.g. 2 hours), and is this consistent across all sessions?
7. How do we handle the non-stationarity introduced by battery ageing? Should we factor in a cycle count / age predictor as a feature, or assume that the model only needs to predict within a short time window where drift is negligible?
8. Are there particular guardrails that feature engineering should be aware of to avoid temporal/session leakage, like whether rolling/lag features can only use past-in-session values?
9. What MAE/RMSE on SoC (%) and time-to-go (in whatever unit, likely minutes) would be considered a successful result for this internship phase?
10. Is the eventual aim of this work to be purely offline/batch processing for the purposes of this internship, or should we be mindful of potential real-time deployment on a BMS later?